决策树 作业1–4

姓名：_____　　学号：_____　　班级：_____

作业1：计算下表中各个属性的信息增益和信息增益率

1. 程序代码（hw1_info_gain.py）

```
# 作业1：计算各个属性的信息增益和信息增益率
from collections import Counter
from math import log2

attrs = ["年龄", "有工作", "有房", "信贷"]
data = [
    # 年龄，有工作，有房，信贷，类别
    ["青年", "否", "否", "一般", "否"],
    ["青年", "否", "否", "好", "否"],
    ["青年", "是", "否", "好", "是"],
    ["青年", "是", "是", "一般", "是"],
    ["青年", "否", "否", "一般", "否"],
    ["中年", "否", "否", "一般", "否"],
    ["中年", "否", "否", "好", "否"],
    ["中年", "是", "是", "好", "是"],
    ["中年", "否", "是", "非常好", "是"],
    ["中年", "否", "是", "非常好", "是"],
    ["老年", "否", "是", "非常好", "是"],
    ["老年", "否", "是", "好", "是"],
    ["老年", "是", "否", "好", "是"],
    ["老年", "是", "否", "非常好", "是"],
    ["老年", "否", "否", "一般", "否"],
]


def pad(s, w):
    """按显示宽度补齐（中文字符宽度为2）"""
    s = str(s)
    return s + " " * (w - sum(2 if ord(ch) > 127 else 1 for ch in s))


def entropy(labels):
    n = len(labels)
    return abs(-sum(c / n * log2(c / n) for c in Counter(labels).values()))


y = [row[-1] for row in data]
H_D = entropy(y)
print(f"样本总数 = {len(data)}，正例(是) = {y.count('是')}，反例(否) = {y.count('否')}")
print(f"数据集信息熵 Ent(D) = {H_D:.4f}\n")

results = []
for i, a in enumerate(attrs):
    col = [row[i] for row in data]
    n = len(col)
    cond = 0.0
    print(f"【{a}】")
    for v, cnt in Counter(col).items():
        sub = [row[-1] for row in data if row[i] == v]
        e = entropy(sub)
        cond += cnt / n * e
        print(f"    {a}={v}：{cnt}个样本（是{sub.count('是')}，否{sub.count('否')}），Ent = {e:.4f}")
    gain = H_D - cond
    iv = entropy(col)
    ratio = gain / iv
    results.append((a, gain, iv, ratio))
    print(f"    条件熵 = {cond:.4f}")
    print(f"    信息增益 Gain(D,{a}) = {H_D:.4f} - {cond:.4f} = {gain:.4f}")
    print(f"    固有值 IV({a}) = {iv:.4f}")
    print(f"    信息增益率 Gain_ratio(D,{a}) = {gain:.4f} / {iv:.4f} = {ratio:.4f}\n")

print("=" * 46)
print(pad("属性", 10) + pad("信息增益", 12) + pad("固有值IV", 12) + "信息增益率")
for a, g, iv, r in results:
```

```
    print(pad(a, 10) + pad(f"{g:.4f}", 12) + pad(f"{iv:.4f}", 12) + f"{r:.4f}")
print("=" * 46)
best_g = max(results, key=lambda t: t[1])
best_r = max(results, key=lambda t: t[3])
print(f"信息增益最大的属性：{best_g[0]}（{best_g[1]:.4f}）")
print(f"信息增益率最大的属性：{best_r[0]}（{best_r[3]:.4f}）")
```

2. 运行结果

终端 — python hw1_info_gain.py

```
$ python hw1_info_gain.py
样本总数 = 15，正例(是) = 9，反例(否) = 6
数据集信息熵 Ent(D) = 0.9710

【年龄】
  年龄=青年：5个样本（是2，否3），Ent = 0.9710
  年龄=中年：5个样本（是3，否2），Ent = 0.9710
  年龄=老年：5个样本（是4，否1），Ent = 0.7219
  条件熵 = 0.8879
  信息增益 Gain(D,年龄) = 0.9710 - 0.8879 = 0.0830
  固有值 IV(年龄) = 1.5850
  信息增益率 Gain_ratio(D,年龄) = 0.0830 / 1.5850 = 0.0524

【有工作】
  有工作=否：10个样本（是4，否6），Ent = 0.9710
  有工作=是：5个样本（是5，否0），Ent = 0.0000
  条件熵 = 0.6473
  信息增益 Gain(D,有工作) = 0.9710 - 0.6473 = 0.3237
  固有值 IV(有工作) = 0.9183
  信息增益率 Gain_ratio(D,有工作) = 0.3237 / 0.9183 = 0.3524

【有房】
  有房=否：9个样本（是3，否6），Ent = 0.9183
  有房=是：6个样本（是6，否0），Ent = 0.0000
  条件熵 = 0.5510
  信息增益 Gain(D,有房) = 0.9710 - 0.5510 = 0.4200
  固有值 IV(有房) = 0.9710
  信息增益率 Gain_ratio(D,有房) = 0.4200 / 0.9710 = 0.4325

【信贷】
  信贷=一般：5个样本（是1，否4），Ent = 0.7219
  信贷=好：6个样本（是4，否2），Ent = 0.9183
  信贷=非常好：4个样本（是4，否0），Ent = 0.0000
  条件熵 = 0.6080
  信息增益 Gain(D,信贷) = 0.9710 - 0.6080 = 0.3630
  固有值 IV(信贷) = 1.5656
  信息增益率 Gain_ratio(D,信贷) = 0.3630 / 1.5656 = 0.2319

==============================================
属性      信息增益    固有值IV    信息增益率
年龄      0.0830      1.5850      0.0524
有工作    0.3237      0.9183      0.3524
有房      0.4200      0.9710      0.4325
信贷      0.3630      1.5656      0.2319
==============================================
信息增益最大的属性：有房（0.4200）
信息增益率最大的属性：有房（0.4325）
```

作业2：计算下表中各个属性的Gini指数

1. 程序代码（hw2_gini.py）

```
# 作业2：计算各个属性的 Gini 指数
from collections import Counter

attrs = ["色泽", "根蒂", "敲声", "纹理", "脐部", "触感"]
data = [
    # 色泽, 根蒂, 敲声, 纹理, 脐部, 触感, 好瓜
    ["青绿", "蜷缩", "浊响", "清晰", "凹陷", "硬滑", "是"],
    ["乌黑", "蜷缩", "沉闷", "清晰", "凹陷", "硬滑", "是"],
    ["乌黑", "蜷缩", "浊响", "清晰", "凹陷", "硬滑", "是"],
    ["青绿", "蜷缩", "沉闷", "清晰", "凹陷", "硬滑", "是"],
    ["浅白", "蜷缩", "浊响", "清晰", "凹陷", "硬滑", "是"],
    ["青绿", "稍蜷", "浊响", "清晰", "稍凹", "软粘", "是"],
    ["乌黑", "稍蜷", "浊响", "稍糊", "稍凹", "软粘", "是"],
    ["乌黑", "稍蜷", "浊响", "清晰", "稍凹", "硬滑", "是"],
    ["乌黑", "稍蜷", "沉闷", "稍糊", "稍凹", "硬滑", "否"],
    ["青绿", "硬挺", "清脆", "清晰", "平坦", "软粘", "否"],
    ["浅白", "硬挺", "清脆", "模糊", "平坦", "硬滑", "否"],
    ["浅白", "蜷缩", "浊响", "模糊", "平坦", "软粘", "否"],
    ["青绿", "稍蜷", "浊响", "稍糊", "凹陷", "硬滑", "否"],
    ["浅白", "稍蜷", "沉闷", "稍糊", "凹陷", "硬滑", "否"],
    ["乌黑", "稍蜷", "浊响", "清晰", "稍凹", "软粘", "否"],
    ["浅白", "蜷缩", "浊响", "模糊", "平坦", "硬滑", "否"],
    ["青绿", "蜷缩", "沉闷", "稍糊", "稍凹", "硬滑", "否"],
]


def pad(s, w):
    """按显示宽度补齐（中文字符宽度为2）"""
    s = str(s)
    return s + " " * (w - sum(2 if ord(ch) > 127 else 1 for ch in s))


def gini(labels):
    n = len(labels)
    return 1 - sum((c / n) ** 2 for c in Counter(labels).values())


y = [row[-1] for row in data]
print(f"样本总数 = {len(data)}, 好瓜 = {y.count('是')}, 坏瓜 = {y.count('否')}")
print(f"数据集基尼值 Gini(D) = 1 - (8/17)^2 - (9/17)^2 = {gini(y):.4f}\n")

results = []
for i, a in enumerate(attrs):
    col = [row[i] for row in data]
    n = len(col)
    gi = 0.0
    terms = []
    print(f"【{a}】")
    for v, cnt in Counter(col).items():
        sub = [row[-1] for row in data if row[i] == v]
        g = gini(sub)
        gi += cnt / n * g
        terms.append(f"{cnt}/{n}×{g:.4f}")
        p, q = sub.count("是"), sub.count("否")
        print(f"    {a}={v}: {cnt}个（是{p}, 否{q}）, "
              f"Gini = 1-({p}/{cnt})^2-({q}/{cnt})^2 = {g:.4f}")
    results.append((a, gi))
    print(f"   Gini_index(D,{a}) = {' + '.join(terms)} = {gi:.4f}\n")

print("=" * 30)
print(pad("属性", 10) + "Gini指数")
for a, g in results:
    print(pad(a, 10) + f"{g:.4f}")
print("=" * 30)
best = min(results, key=lambda t: t[1])
print(f"Gini指数最小的属性：{best[0]}（{best[1]:.4f}），应选为最优划分属性")
```

2. 运行结果

```
● ● ●  终端 — python hw2_gini.py

$ python hw2_gini.py
样本总数 = 17, 好瓜 = 8, 坏瓜 = 9
数据集基尼值 Gini(D) = 1 - (8/17)^2 - (9/17)^2 = 0.4983

【色泽】
  色泽=青绿: 6个（是3, 否3）, Gini = 1-(3/6)^2-(3/6)^2 = 0.5000
  色泽=乌黑: 6个（是4, 否2）, Gini = 1-(4/6)^2-(2/6)^2 = 0.4444
  色泽=浅白: 5个（是1, 否4）, Gini = 1-(1/5)^2-(4/5)^2 = 0.3200
  Gini_index(D,色泽) = 6/17×0.5000 + 6/17×0.4444 + 5/17×0.3200 = 0.4275

【根蒂】
  根蒂=蜷缩: 8个（是5, 否3）, Gini = 1-(5/8)^2-(3/8)^2 = 0.4688
  根蒂=稍蜷: 7个（是3, 否4）, Gini = 1-(3/7)^2-(4/7)^2 = 0.4898
  根蒂=硬挺: 2个（是0, 否2）, Gini = 1-(0/2)^2-(2/2)^2 = 0.0000
  Gini_index(D,根蒂) = 8/17×0.4688 + 7/17×0.4898 + 2/17×0.0000 = 0.4223

【敲声】
  敲声=浊响: 10个（是6, 否4）, Gini = 1-(6/10)^2-(4/10)^2 = 0.4800
  敲声=沉闷: 5个（是2, 否3）, Gini = 1-(2/5)^2-(3/5)^2 = 0.4800
  敲声=清脆: 2个（是0, 否2）, Gini = 1-(0/2)^2-(2/2)^2 = 0.0000
  Gini_index(D,敲声) = 10/17×0.4800 + 5/17×0.4800 + 2/17×0.0000 = 0.4235

【纹理】
  纹理=清晰: 9个（是7, 否2）, Gini = 1-(7/9)^2-(2/9)^2 = 0.3457
  纹理=稍糊: 5个（是1, 否4）, Gini = 1-(1/5)^2-(4/5)^2 = 0.3200
  纹理=模糊: 3个（是0, 否3）, Gini = 1-(0/3)^2-(3/3)^2 = 0.0000
  Gini_index(D,纹理) = 9/17×0.3457 + 5/17×0.3200 + 3/17×0.0000 = 0.2771

【脐部】
  脐部=凹陷: 7个（是5, 否2）, Gini = 1-(5/7)^2-(2/7)^2 = 0.4082
  脐部=稍凹: 6个（是3, 否3）, Gini = 1-(3/6)^2-(3/6)^2 = 0.5000
  脐部=平坦: 4个（是0, 否4）, Gini = 1-(0/4)^2-(4/4)^2 = 0.0000
  Gini_index(D,脐部) = 7/17×0.4082 + 6/17×0.5000 + 4/17×0.0000 = 0.3445

【触感】
  触感=硬滑: 12个（是6, 否6）, Gini = 1-(6/12)^2-(6/12)^2 = 0.5000
  触感=软粘: 5个（是2, 否3）, Gini = 1-(2/5)^2-(3/5)^2 = 0.4800
  Gini_index(D,触感) = 12/17×0.5000 + 5/17×0.4800 = 0.4941

============================
属性      Gini指数
色泽      0.4275
根蒂      0.4223
敲声      0.4235
纹理      0.2771
脐部      0.3445
触感      0.4941
============================
Gini指数最小的属性：纹理（0.2771）, 应选为最优划分属性
```

作业3：用信息熵进行划分选择的方法，为下表生成决策树

1. 程序代码（hw3_id3_tree.py）

```
# 作业3：用信息熵（信息增益）进行划分选择，为西瓜数据集生成决策树
# 离散属性多路划分；连续属性“含糖量”采用二分法（候选划分点为相邻取值的中点）
from collections import Counter
from math import log2
import json

attrs = ["色泽", "根蒂", "敲声", "纹理", "触感", "含糖量"]
continuous = {"含糖量"}
data = [
    # 色泽, 根蒂, 敲声, 纹理, 触感, 含糖量, 好瓜
    ["青绿", "蜷缩", "浊响", "清晰", "硬滑", 0.460, "是"],
    ["乌黑", "蜷缩", "沉闷", "清晰", "硬滑", 0.376, "是"],
    ["乌黑", "蜷缩", "浊响", "清晰", "硬滑", 0.264, "是"],
    ["青绿", "蜷缩", "沉闷", "清晰", "硬滑", 0.318, "是"],
    ["浅白", "蜷缩", "浊响", "清晰", "硬滑", 0.215, "是"],
    ["青绿", "稍蜷", "浊响", "清晰", "软粘", 0.237, "是"],
    ["乌黑", "稍蜷", "浊响", "稍糊", "软粘", 0.149, "是"],
    ["乌黑", "稍蜷", "浊响", "清晰", "硬滑", 0.211, "是"],
    ["乌黑", "稍蜷", "沉闷", "稍糊", "硬滑", 0.091, "否"],
    ["青绿", "硬挺", "清脆", "清晰", "软粘", 0.267, "否"],
    ["浅白", "硬挺", "清脆", "模糊", "硬滑", 0.057, "否"],
    ["浅白", "蜷缩", "浊响", "模糊", "软粘", 0.099, "否"],
    ["青绿", "稍蜷", "浊响", "稍糊", "硬滑", 0.161, "否"],
    ["浅白", "稍蜷", "沉闷", "稍糊", "硬滑", 0.198, "否"],
    ["乌黑", "稍蜷", "浊响", "清晰", "软粘", 0.370, "否"],
    ["浅白", "蜷缩", "浊响", "模糊", "硬滑", 0.042, "否"],
    ["青绿", "蜷缩", "沉闷", "稍糊", "硬滑", 0.103, "否"],
]
values = {a: list(dict.fromkeys(r[i] for r in data)) for i, a in enumerate(attrs) if a not in continuous}


def entropy(rows):
    n = len(rows)
    if n == 0:
        return 0.0
    return abs(-sum(c / n * log2(c / n) for c in Counter(r[-1] for r in rows).values()))


def majority(rows):
    cnt = Counter(r[-1] for r in rows)
    return "是" if cnt["是"] >= cnt["否"] else "否"


def gain(rows, a):
    """返回（信息增益，划分点）；离散属性划分点为 None"""
    i = attrs.index(a)
    base, n = entropy(rows), len(rows)
    if a in continuous:
        vs = sorted({r[i] for r in rows})
        best = (-1.0, None)
        for t in [(vs[k] + vs[k + 1]) / 2 for k in range(len(vs) - 1)]:
            left = [r for r in rows if r[i] <= t]
            right = [r for r in rows if r[i] > t]
            g = base - len(left) / n * entropy(left) - len(right) / n * entropy(right)
            if g > best[0] + 1e-12:
                best = (g, t)
        return best
    g = base
    for v in values[a]:
        sub = [r for r in rows if r[i] == v]
        g -= len(sub) / n * entropy(sub)
    return g, None


def build(rows, avail, depth=0, cond="根节点"):
    ind = "    " * depth
    labels = Counter(r[-1] for r in rows)
    print(f"{ind}▶ {cond}：样本{len(rows)}个（是{labels['是']}, 否{labels['否']}），"
          f"Ent = {entropy(rows):.4f}")
    if len(labels) == 1:
```

```
        print(f"{ind}  全部属于同一类 → 叶节点：{rows[0][-1]}")
        return rows[0][-1]
    usable = [a for a in avail if a in continuous or len({r[attrs.index(a)] for r in rows}) > 1]
    if not usable:
        print(f"{ind}  属性已用完 → 叶节点：{majority(rows)}")
        return majority(rows)
    scores = {a: gain(rows, a) for a in usable}
    for a in usable:
        g, t = scores[a]
        extra = f"（划分点 {t:.4f}）" if t is not None else ""
        print(f"{ind}  Gain({a}) = {g:.4f}{extra}")
    best = max(usable, key=lambda a: scores[a][0])
    g, t = scores[best]
    i = attrs.index(best)
    if best in continuous:
        print(f"{ind}  ⇒ 选择 {best} ≤ {t:.4f} 进行划分")
        node = {"属性": f"{best}≤{t:.4f}?", "分支": {}}
        for name, sub in (("是", [r for r in rows if r[i] <= t]), ("否", [r for r in rows if r[i] > t])):
            node["分支"][name] = build(sub, avail, depth + 1, f"{best}≤{t:.4f} = {name}")
        return node
    print(f"{ind}  ⇒ 选择 {best} 进行划分")
    node = {"属性": best, "分支": {}}
    rest = [a for a in avail if a != best]
    for v in values[best]:
        sub = [r for r in rows if r[i] == v]
        if not sub:
            print(f"{ind}    ▶ {best}={v}：无样本 → 叶节点（父节点多数类）：{majority(rows)}")
            node["分支"][v] = majority(rows)
        else:
            node["分支"][v] = build(sub, rest, depth + 1, f"{best}={v}")
    return node


def show(node, prefix=""):
    """以文本树形式输出最终决策树"""
    items = list(node["分支"].items())
    for k, (v, child) in enumerate(items):
        last = k == len(items) - 1
        branch = " └──── " if last else " ├──── "
        if isinstance(child, dict):
            print(f"{prefix}{branch}{v} → [{child['属性']}]")
            show(child, prefix + ("      " if last else " │    "))
        else:
            print(f"{prefix}{branch}{v} → {'好瓜' if child == '是' else '坏瓜'}")


def predict(node, row):
    while isinstance(node, dict):
        a = node["属性"]
        if a.endswith("?"):
            name, t = a[:-1].split("≤")
            node = node["分支"]["是" if row[attrs.index(name)] <= float(t) else "否"]
        else:
            node = node["分支"][row[attrs.index(a)]]
    return node


print("========== 决策树生成过程 ==========")
tree = build(data, attrs)
print("\n========== 最终决策树 ==========")
print(f"[{tree['属性']}]")
show(tree)
acc = sum(predict(tree, r) == r[-1] for r in data) / len(data)
print(f"\n训练集精度：{acc:.2%}")
json.dump(tree, open("hw3_tree.json", "w", encoding="utf-8"), ensure_ascii=False, indent=2)
```

2. 运行结果

```
终端 — python hw3_id3_tree.py

$ python hw3_id3_tree.py
========== 决策树生成过程 ==========
▶ 根节点：样本17个（是8, 否9），Ent = 0.9975
  Gain(色泽) = 0.1081
```

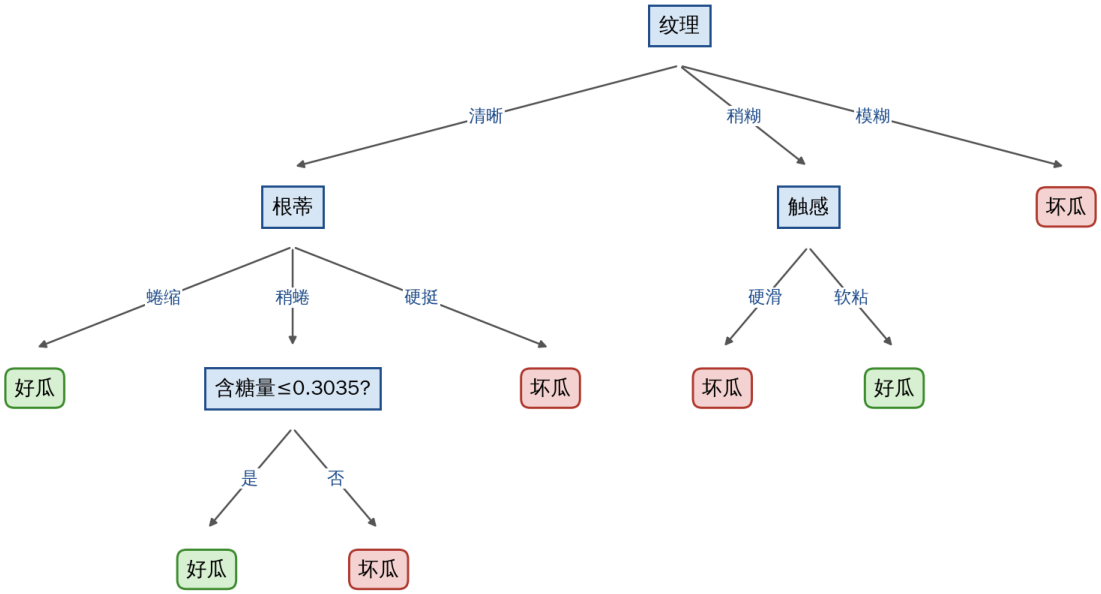
```
  Gain(根蒂) = 0.1427
  Gain(敲声) = 0.1408
  Gain(纹理) = 0.3806
  Gain(触感) = 0.0060
  Gain(含糖量) = 0.3493（划分点 0.1260）
  ⇒ 选择 纹理 进行划分
    ▶ 纹理=清晰：样本9个（是7, 否2）, Ent = 0.7642
      Gain(色泽) = 0.0431
      Gain(根蒂) = 0.4581
      Gain(敲声) = 0.3309
      Gain(触感) = 0.4581
      Gain(含糖量) = 0.2248（划分点 0.2655）
      ⇒ 选择 根蒂 进行划分
        ▶ 根蒂=蜷缩：样本5个（是5, 否0）, Ent = 0.0000
          全部属于同一类 → 叶节点：是
        ▶ 根蒂=稍蜷：样本3个（是2, 否1）, Ent = 0.9183
          Gain(色泽) = 0.2516
          Gain(触感) = 0.2516
          Gain(含糖量) = 0.9183（划分点 0.3035）
          ⇒ 选择 含糖量 ≤ 0.3035 进行划分
            ▶ 含糖量≤0.3035 = 是：样本2个（是2, 否0）, Ent = 0.0000
              全部属于同一类 → 叶节点：是
            ▶ 含糖量≤0.3035 = 否：样本1个（是0, 否1）, Ent = 0.0000
              全部属于同一类 → 叶节点：否
        ▶ 根蒂=硬挺：样本1个（是0, 否1）, Ent = 0.0000
          全部属于同一类 → 叶节点：否
    ▶ 纹理=稍糊：样本5个（是1, 否4）, Ent = 0.7219
      Gain(色泽) = 0.3219
      Gain(根蒂) = 0.0729
      Gain(敲声) = 0.3219
      Gain(触感) = 0.7219
      Gain(含糖量) = 0.1710（划分点 0.1260）
      ⇒ 选择 触感 进行划分
        ▶ 触感=硬滑：样本4个（是0, 否4）, Ent = 0.0000
          全部属于同一类 → 叶节点：否
        ▶ 触感=软粘：样本1个（是1, 否0）, Ent = 0.0000
          全部属于同一类 → 叶节点：是
    ▶ 纹理=模糊：样本3个（是0, 否3）, Ent = 0.0000
      全部属于同一类 → 叶节点：否

========== 最终决策树 ==========
[纹理]
├─── 清晰 → [根蒂]
│    ├─── 蜷缩 → 好瓜
│    ├─── 稍蜷 → [含糖量≤0.3035?]
│    │    ├─── 是 → 好瓜
│    │    └─── 否 → 坏瓜
│    └─── 硬挺 → 坏瓜
├─── 稍糊 → [触感]
│    ├─── 硬滑 → 坏瓜
│    └─── 软粘 → 好瓜
└─── 模糊 → 坏瓜

训练集精度：100.00%
```

3. 决策树

作业3：基于信息增益的决策树



作业4：用基尼指数进行划分方法，生成预剪枝、后剪枝决策树，并与未剪枝决策树（基于信息增益、训练集生成）进行比较

1. 程序代码（hw4_pruning.py）

```
# 作业4：用基尼指数进行划分，生成预剪枝、后剪枝决策树，
#        并与未剪枝决策树（基于信息增益、训练集生成）进行比较
# 离散属性多路划分；连续属性“密度”采用二分法；
# 剪枝判断依据为验证集精度：只有精度严格提高时才划分（预剪枝）/剪枝（后剪枝）
from collections import Counter
from math import log2
import copy
import json

attrs = ["色泽", "根蒂", "敲声", "脐部", "密度"]
continuous = {"密度"}
train = [
    # 编号，色泽，根蒂，敲声，脐部，密度，好瓜
    [1, "青绿", "蜷缩", "浊响", "凹陷", 0.697, "是"],
    [2, "乌黑", "蜷缩", "沉闷", "凹陷", 0.774, "是"],
    [3, "乌黑", "蜷缩", "浊响", "凹陷", 0.634, "是"],
    [6, "青绿", "稍蜷", "浊响", "稍凹", 0.403, "是"],
    [7, "乌黑", "稍蜷", "浊响", "稍凹", 0.481, "是"],
    [10, "青绿", "硬挺", "清脆", "平坦", 0.234, "否"],
    [14, "浅白", "稍蜷", "沉闷", "凹陷", 0.657, "否"],
    [15, "乌黑", "稍蜷", "浊响", "稍凹", 0.360, "否"],
    [16, "浅白", "蜷缩", "浊响", "平坦", 0.593, "否"],
    [17, "青绿", "蜷缩", "沉闷", "稍凹", 0.719, "否"],
]
valid = [
    [4, "青绿", "蜷缩", "沉闷", "凹陷", 0.608, "是"],
    [5, "浅白", "蜷缩", "浊响", "凹陷", 0.556, "是"],
    [8, "乌黑", "稍蜷", "浊响", "稍凹", 0.437, "是"],
    [9, "乌黑", "稍蜷", "沉闷", "稍凹", 0.666, "否"],
    [11, "浅白", "硬挺", "清脆", "平坦", 0.245, "否"],
    [12, "浅白", "蜷缩", "浊响", "平坦", 0.343, "否"],
    [13, "青绿", "稍蜷", "浊响", "凹陷", 0.639, "否"],
]
col = {a: i + 1 for i, a in enumerate(attrs)}
# 多个属性得分相同时的优先顺序（与教材《机器学习》4.3节一致，优先选择“脐部”）
priority = ["脐部", "色泽", "根蒂", "敲声", "密度"]
values = {a: list(dict.fromkeys(r[col[a]] for r in train + valid)) for a in attrs if a not in continuous}


# ---------------- 基本度量 ----------------
def gini(rows):
    n = len(rows)
    return 1 - sum((c / n) ** 2 for c in Counter(r[-1] for r in rows).values()) if n else 0.0


def entropy(rows):
    n = len(rows)
    return abs(-sum(c / n * log2(c / n) for c in Counter(r[-1] for r in rows).values())) if n else 0.0


def majority(rows):
    cnt = Counter(r[-1] for r in rows)
    return "是" if cnt["是"] >= cnt["否"] else "否"


def split(rows, a, t=None):
    """按属性 a 划分，返回 [(分支名，子集)]"""
    i = col[a]
    if a in continuous:
        return [(f"≤{t:.4f}", [r for r in rows if r[i] <= t]),
                (f">{t:.4f}", [r for r in rows if r[i] > t])]
    return [(v, [r for r in rows if r[i] == v]) for v in values[a]]


def evaluate(rows, a, criterion):
    """返回（得分，划分点），得分越大越好：信息增益 或 负的基尼指数"""
    n = len(rows)
    imp = entropy if criterion == "gain" else gini
```

```
    def score(parts):
        s = sum(len(sub) / n * imp(sub) for _, sub in parts)
        return entropy(rows) - s if criterion == "gain" else -s

    if a in continuous:
        vs = sorted({r[col[a]] for r in rows})
        best = (float("-inf"), None)
        for t in [(vs[k] + vs[k + 1]) / 2 for k in range(len(vs) - 1)]:
            s = score(split(rows, a, t))
            if s > best[0] + 1e-12:
                best = (s, t)
        return best
    return score(split(rows, a)), None


def choose(rows, avail, criterion, ind=""):
    usable = [a for a in avail if len({r[col[a]] for r in rows}) > 1]
    if not usable:
        return None, None
    scores = {a: evaluate(rows, a, criterion) for a in usable}
    name = "Gain" if criterion == "gain" else "Gini_index"
    print(ind + "  " + ", ".join(
        f"{name}({a})={abs(s):.4f}" + (f"[t={t:.4f}]" if t is not None else "")
        for a, (s, t) in scores.items()))
    best = max(usable, key=lambda a: (round(scores[a][0], 10), -priority.index(a)))
    return best, scores[best][1]


# ---------------- 树结构 ----------------
def leaf(rows, parent_rows=None):
    return {"叶": majority(rows if rows else parent_rows), "rows": rows or []}


def predict(node, r):
    while "叶" not in node:
        a, t = node["属性"], node["t"]
        if a in continuous:
            node = node["分支"][f"≤{t:.4f}" if r[col[a]] <= t else f">{t:.4f}"]
        else:
            node = node["分支"][r[col[a]]]
    return node["叶"]


def accuracy(tree, rows):
    return sum(predict(tree, r) == r[-1] for r in rows) / len(rows)


def count(node):
    """返回 (节点总数, 叶节点数, 深度)"""
    if "叶" in node:
        return 1, 1, 0
    stats = [count(c) for c in node["分支"].values()]
    return 1 + sum(s[0] for s in stats), sum(s[1] for s in stats), 1 + max(s[2] for s in stats)


def label(node):
    a = node["属性"]
    return f"{a}≤{node['t']:.4f}?" if a in continuous else a


def show(node, prefix=""):
    items = list(node["分支"].items())
    for k, (v, c) in enumerate(items):
        last = k == len(items) - 1
        br = " └──── " if last else " ├──── "
        if "叶" in c:
            print(f"{prefix}{br}{v} → {'好瓜' if c['叶'] == '是' else '坏瓜'}")
        else:
            print(f"{prefix}{br}{v} → [{label(c)}]")
            show(c, prefix + ("      " if last else " |    "))


def print_tree(tree):
    if "叶" in tree:
        print(f" (单叶节点) → {'好瓜' if tree['叶'] == '是' else '坏瓜'}")
```

```
    else:
        print(f"[{label(tree)}]")
        show(tree)


def to_json(node):
    if "叶" in node:
        return node["叶"]
    return {"属性": label(node), "分支": {k: to_json(c) for k, c in node["分支"].items()}}


def ids(rows):
    return "{" + ",".join(str(r[0]) for r in rows) + "}"


# ---------------- 生成完整（未剪枝）树 ----------------
def build_full(rows, avail, criterion, depth=0, cond="根节点"):
    ind = "    " * depth
    print(f"{ind}▶ {cond}：{ids(rows)}")
    if len({r[-1] for r in rows}) == 1:
        print(f"{ind}  → 叶节点：{rows[0][-1]}")
        return leaf(rows)
    a, t = choose(rows, avail, criterion, ind)
    if a is None:
        print(f"{ind}  → 无可用属性，叶节点：{majority(rows)}")
        return leaf(rows)
    print(f"{ind}  ⇒ 选择 {a}" + (f" ≤ {t:.4f}" if t is not None else ""))
    rest = avail if a in continuous else [x for x in avail if x != a]
    node = {"属性": a, "t": t, "rows": rows, "分支": {}}
    for v, sub in split(rows, a, t):
        if sub:
            cond = f"{a}{v}" if a in continuous else f"{a}={v}"
            node["分支"][v] = build_full(sub, rest, criterion, depth + 1, cond)
        else:
            print(f"{ind}    ▶ {a}={v}：无训练样本 → 叶节点（父节点多数类）：{majority(rows)}")
            node["分支"][v] = leaf([], rows)
    return node


# ---------------- 预剪枝 ----------------
def build_prepruned():
    root = leaf(train)
    root["avail"] = attrs
    queue = [(root, None, None)]  # (节点, 父节点, 分支名)，广度优先
    tree = root
    while queue:
        node, parent, key = queue.pop(0)
        rows = node["rows"]
        print(f"▶ 节点 {ids(rows)}（当前标记为“{node['叶']}”）")
        if len({r[-1] for r in rows}) == 1:
            print("  样本全属于同一类，不再划分\n")
            continue
        a, t = choose(rows, node["avail"], "gini")
        if a is None:
            print("  无可用属性，不再划分\n")
            continue
        before = accuracy(tree, valid)
        rest = node["avail"] if a in continuous else [x for x in node["avail"] if x != a]
        new = {"属性": a, "t": t, "rows": rows, "分支": {}}
        for v, sub in split(rows, a, t):
            child = leaf(sub, rows)
            child["avail"] = rest
            new["分支"][v] = child
        cand = new if parent is None else tree
        if parent is not None:
            parent["分支"][key] = new
        after = accuracy(cand, valid)
        name = f"{a}" + (f"≤{t:.4f}" if t is not None else "")
        print(f"  按“{name}”划分：划分前验证集精度 {before:.2%}，划分后 {after:.2%}", end="")
        if after > before + 1e-12:
            print(" → 精度提高，进行划分\n")
            tree = cand
            for v, c in new["分支"].items():
                if c["rows"]:
                    queue.append((c, new, v))
        else:
```

```
            print(" → 精度未提高，禁止划分（剪枝）\n")
            if parent is not None:
                parent["分支"][key] = node
    return tree


# ---------------- 后剪枝 ----------------
def postprune(tree):
    def internal_nodes(node, parent=None, key=None, depth=0, out=None):
        out = [] if out is None else out
        if "叶" not in node:
            out.append((depth, node, parent, key))
            for k, c in node["分支"].items():
                internal_nodes(c, node, k, depth + 1, out)
        return out

    nodes = internal_nodes(tree)
    nodes.sort(key=lambda x: -x[0])  # 自底向上：从最深的内部节点开始考察
    for depth, node, parent, key in nodes:
        before = accuracy(tree, valid)
        lf = leaf(node["rows"])
        if parent is None:
            after = accuracy(lf, valid)
        else:
            parent["分支"][key] = lf
            after = accuracy(tree, valid)
        name = label(node)
        print(f"▶ 考察节点[{name}] {ids(node['rows'])}：剪枝前验证集精度 {before:.2%}, "
              f"替换为叶节点“{lf['叶']}”后 {after:.2%}", end="")
        if after > before + 1e-12:
            print(" → 精度提高，剪枝")
            if parent is None:
                tree = lf
        else:
            print(" → 精度未提高，保留")
            if parent is not None:
                parent["分支"][key] = node
    return tree


print(f"训练集：{ids(train)}  验证集：{ids(valid)}\n")

print("=" * 18 + " 一、未剪枝决策树（信息增益，训练集） " + "=" * 18)
tree_gain = build_full(train, attrs, "gain")
print()
print_tree(tree_gain)

print("\n" + "=" * 18 + " 二、未剪枝决策树（基尼指数，训练集） " + "=" * 18)
tree_gini = build_full(train, attrs, "gini")
print()
print_tree(tree_gini)

print("\n" + "=" * 22 + " 三、预剪枝决策树（基尼指数） " + "=" * 22)
tree_pre = build_prepruned()
print_tree(tree_pre)

print("\n" + "=" * 22 + " 四、后剪枝决策树（基尼指数） " + "=" * 22)
tree_post = postprune(copy.deepcopy(tree_gini))
print()
print_tree(tree_post)


def pad(s, w):
    s = str(s)
    return s + " " * (w - sum(2 if ord(ch) > 127 else 1 for ch in s))


print("\n" + "=" * 28 + " 五、比较 " + "=" * 28)
print(pad("决策树", 30) + pad("节点数", 8) + pad("叶节点", 8) + pad("深度", 6)
      + pad("训练集精度", 12) + "验证集精度")
for name, t in [("未剪枝（信息增益）", tree_gain), ("未剪枝（基尼指数）", tree_gini),
                ("预剪枝（基尼指数）", tree_pre), ("后剪枝（基尼指数）", tree_post)]:
    n, l, d = count(t)
    print(pad(name, 30) + pad(n, 8) + pad(l, 8) + pad(d, 6)
          + pad(f"{accuracy(t, train):.2%}", 12) + f"{accuracy(t, valid):.2%}")
```

```
print("\n结论：")
print("1. 未剪枝决策树在训练集上精度为100%，但验证集精度只有"
      f"{accuracy(tree_gain, valid):.2%}，存在过拟合；")
print(f"2. 预剪枝决策树只有{count(tree_pre)[0]}个节点，验证集精度提高到{accuracy(tree_pre, valid):.2%}，"
      "训练时间开销小，但有欠拟合的风险；")
print(f"3. 后剪枝决策树保留了更多分支（{count(tree_post)[0]}个节点），验证集精度为{accuracy(tree_post, valid):.2%}，"
      "欠拟合风险小，泛化性能一般优于预剪枝，但训练时间开销更大。")

json.dump({"未剪枝（信息增益）": to_json(tree_gain), "未剪枝（基尼指数）": to_json(tree_gini),
           "预剪枝（基尼指数）": to_json(tree_pre), "后剪枝（基尼指数）": to_json(tree_post)},
          open("hw4_trees.json", "w", encoding="utf-8"), ensure_ascii=False, indent=2)
```

2. 运行结果

```
终端 — python hw4_pruning.py

$ python hw4_pruning.py
训练集：{1,2,3,6,7,10,14,15,16,17}   验证集：{4,5,8,9,11,12,13}

================== 一、未剪枝决策树（信息增益，训练集） ==================
▶ 根节点：{1,2,3,6,7,10,14,15,16,17}
  Gain(色泽)=0.2755, Gain(根蒂)=0.1145, Gain(敲声)=0.1735, Gain(脐部)=0.2755, Gain(密度)=0.2365[t=0.3815]
  ⇒ 选择 脐部
    ▶ 脐部=凹陷：{1,2,3,14}
      Gain(色泽)=0.8113, Gain(根蒂)=0.8113, Gain(敲声)=0.3113, Gain(密度)=0.3113[t=0.6770]
      ⇒ 选择 色泽
        ▶ 色泽=青绿：{1}
          → 叶节点：是
        ▶ 色泽=乌黑：{2,3}
          → 叶节点：是
        ▶ 色泽=浅白：{14}
          → 叶节点：否
    ▶ 脐部=稍凹：{6,7,15,17}
      Gain(色泽)=0.0000, Gain(根蒂)=0.3113, Gain(敲声)=0.3113, Gain(密度)=0.3113[t=0.3815]
      ⇒ 选择 根蒂
        ▶ 根蒂=蜷缩：{17}
          → 叶节点：否
        ▶ 根蒂=稍蜷：{6,7,15}
          Gain(色泽)=0.2516, Gain(密度)=0.9183[t=0.3815]
          ⇒ 选择 密度 ≤ 0.3815
            ▶ 密度≤0.3815：{15}
              → 叶节点：否
            ▶ 密度>0.3815：{6,7}
              → 叶节点：是
        ▶ 根蒂=硬挺：无训练样本 → 叶节点（父节点多数类）：是
    ▶ 脐部=平坦：{10,16}
      → 叶节点：否

[脐部]
├── 凹陷 → [色泽]
│   ├── 青绿 → 好瓜
│   ├── 乌黑 → 好瓜
│   └── 浅白 → 坏瓜
├── 稍凹 → [根蒂]
│   ├── 蜷缩 → 坏瓜
│   ├── 稍蜷 → [密度≤0.3815?]
│   │   ├── ≤0.3815 → 坏瓜
│   │   └── >0.3815 → 好瓜
│   └── 硬挺 → 好瓜
└── 平坦 → 坏瓜

================== 二、未剪枝决策树（基尼指数，训练集） ==================
▶ 根节点：{1,2,3,6,7,10,14,15,16,17}
  Gini_index(色泽)=0.3500, Gini_index(根蒂)=0.4400, Gini_index(敲声)=0.4000, Gini_index(脐部)=0.3500, Gini_index(密
度)=0.3750[t=0.3815]
  ⇒ 选择 脐部
    ▶ 脐部=凹陷：{1,2,3,14}
      Gini_index(色泽)=0.0000, Gini_index(根蒂)=0.0000, Gini_index(敲声)=0.2500, Gini_index(密度)=0.2500[t=0.6770]
      ⇒ 选择 色泽
        ▶ 色泽=青绿：{1}
          → 叶节点：是
        ▶ 色泽=乌黑：{2,3}
          → 叶节点：是
        ▶ 色泽=浅白：{14}
```

```
          → 叶节点：否
    ▶ 脐部=稍凹：{6,7,15,17}
      Gini_index(色泽)=0.5000, Gini_index(根蒂)=0.3333, Gini_index(敲声)=0.3333, Gini_index(密度)=0.3333[t=0.3815]
      ⇒ 选择 根蒂
        ▶ 根蒂=蜷缩：{17}
          → 叶节点：否
        ▶ 根蒂=稍蜷：{6,7,15}
          Gini_index(色泽)=0.3333, Gini_index(密度)=0.0000[t=0.3815]
          ⇒ 选择 密度 ≤ 0.3815
            ▶ 密度≤0.3815：{15}
              → 叶节点：否
            ▶ 密度>0.3815：{6,7}
              → 叶节点：是
        ▶ 根蒂=硬挺：无训练样本 → 叶节点（父节点多数类）：是
    ▶ 脐部=平坦：{10,16}
      → 叶节点：否

[脐部]
├─── 凹陷 → [色泽]
│    ├─── 青绿 → 好瓜
│    ├─── 乌黑 → 好瓜
│    └─── 浅白 → 坏瓜
├─── 稍凹 → [根蒂]
│    ├─── 蜷缩 → 坏瓜
│    ├─── 稍蜷 → [密度≤0.3815?]
│    │    ├─── ≤0.3815 → 坏瓜
│    │    └─── >0.3815 → 好瓜
│    └─── 硬挺 → 好瓜
└─── 平坦 → 坏瓜

====================== 三、预剪枝决策树（基尼指数） ======================
▶ 节点 {1,2,3,6,7,10,14,15,16,17}（当前标记为"是"）
  Gini_index(色泽)=0.3500, Gini_index(根蒂)=0.4400, Gini_index(敲声)=0.4000, Gini_index(脐部)=0.3500, Gini_index(密
度)=0.3750[t=0.3815]
  按"脐部"划分：划分前验证集精度 42.86%，划分后 71.43% → 精度提高，进行划分

▶ 节点 {1,2,3,14}（当前标记为"是"）
  Gini_index(色泽)=0.0000, Gini_index(根蒂)=0.0000, Gini_index(敲声)=0.2500, Gini_index(密度)=0.2500[t=0.6770]
  按"色泽"划分：划分前验证集精度 71.43%，划分后 57.14% → 精度未提高，禁止划分（剪枝）

▶ 节点 {6,7,15,17}（当前标记为"是"）
  Gini_index(色泽)=0.5000, Gini_index(根蒂)=0.3333, Gini_index(敲声)=0.3333, Gini_index(密度)=0.3333[t=0.3815]
  按"根蒂"划分：划分前验证集精度 71.43%，划分后 71.43% → 精度未提高，禁止划分（剪枝）

▶ 节点 {10,16}（当前标记为"否"）
  样本全属于同一类，不再划分

[脐部]
├─── 凹陷 → 好瓜
├─── 稍凹 → 好瓜
└─── 平坦 → 坏瓜

====================== 四、后剪枝决策树（基尼指数） ======================
▶ 考察节点[密度≤0.3815?] {6,7,15}：剪枝前验证集精度 57.14%，替换为叶节点"是"后 57.14% → 精度未提高，保留
▶ 考察节点[色泽] {1,2,3,14}：剪枝前验证集精度 57.14%，替换为叶节点"是"后 71.43% → 精度提高，剪枝
▶ 考察节点[根蒂] {6,7,15,17}：剪枝前验证集精度 71.43%，替换为叶节点"是"后 71.43% → 精度未提高，保留
▶ 考察节点[脐部] {1,2,3,6,7,10,14,15,16,17}：剪枝前验证集精度 71.43%，替换为叶节点"是"后 42.86% → 精度未提高，保留

[脐部]
├─── 凹陷 → 好瓜
├─── 稍凹 → [根蒂]
│    ├─── 蜷缩 → 坏瓜
│    ├─── 稍蜷 → [密度≤0.3815?]
│    │    ├─── ≤0.3815 → 坏瓜
│    │    └─── >0.3815 → 好瓜
│    └─── 硬挺 → 好瓜
└─── 平坦 → 坏瓜

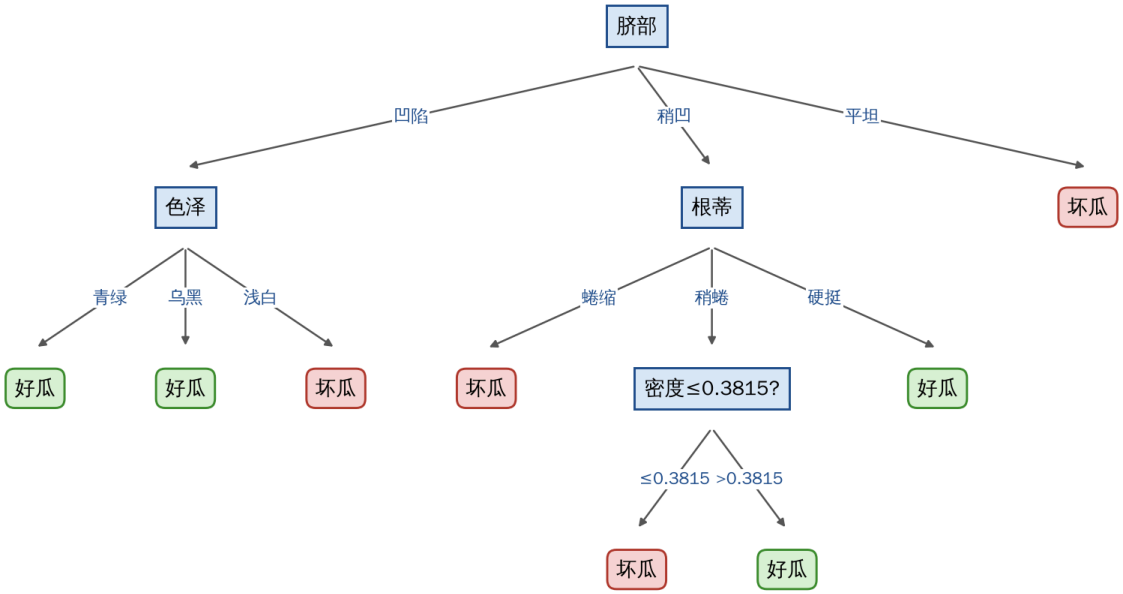
========================== 五、比较 ==========================
决策树                        节点数  叶节点  深度  训练集精度  验证集精度
未剪枝（信息增益）            12      8       3     100.00%     57.14%
未剪枝（基尼指数）            12      8       3     100.00%     57.14%
预剪枝（基尼指数）            4       3       1     70.00%      71.43%
后剪枝（基尼指数）            9       6       3     90.00%      71.43%

结论：
```

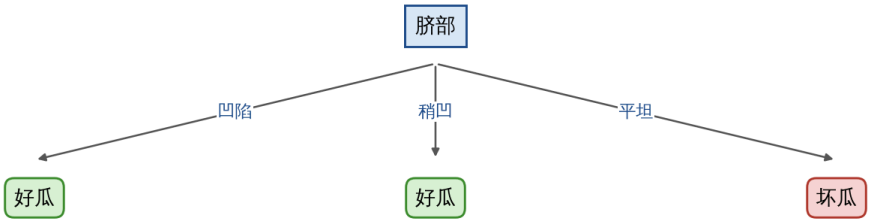
1. 未剪枝决策树在训练集上精度为100%，但验证集精度只有57.14%，存在过拟合；
2. 预剪枝决策树只有4个节点，验证集精度提高到71.43%，训练时间开销小，但有欠拟合的风险；
3. 后剪枝决策树保留了更多分支（9个节点），验证集精度为71.43%，欠拟合风险小，泛化性能一般优于预剪枝，但训练时间开销更大。

3. 决策树

作业4：未剪枝（信息增益）决策树



作业4：预剪枝（基尼指数）决策树



作业4：后剪枝（基尼指数）决策树

